A Systematic Benchmark of Supervised and Unsupervised Graph Embedding Methods

Konstantina Koulaktsidou
Department of Electrical and Computer Engineering
National Technical University of Athens
email

Fragiska Kiminou
Department of Electrical and Computer Engineering
National Technical University of Athens
email

Nikolaos Moraitis
Department of Electrical and Computer Engineering
National Technical University of Athens
email

***Abstract*—Graph embedding methods provide low dimensional representations of graphs for downstream learning tasks. In this work, we compare unsupervised and supervised graph embedding techniques on real world benchmark datasets. We evaluate Graph2Vec, NetLSD and the Graph Isomorphism Network (GIN) on MUTAG, ENZYMES and IMDB-MULTI using graph classification and clustering, considering accuracy, computational efficiency and scalability. Results show that GIN achieves the highest classification accuracy at a higher computational cost, while Graph2Vec offers a favorable balance between performance and efficiency. NetLSD demonstrates competitive clustering performance with minimal overhead. Overall, each method shows distinct trade offs and maintains reasonable robustness under structural perturbations providing practical guidance for method selection under different data and computational constraints.**

***Index Terms*—Graph2Vec, NetLSD, GIN, Graph Embedding, Graph Classification, Graph Clustering**

I. Introduction

Graphs are widely uses to model relationship and interactions in data from multiple domains such as network analysis, bioinformatics and chemistry. Many learning tasks are defined at the graph level. However, graphs have irregular structure with different variable sizes, features that make them face challenges to traditional machine learning algorithms.

Graph embedding methods address this limitation by mapping entire graphs into fixed size vector representations and this helps the ML techniques to work efficiently to downstream tasks. Early approaches relied on handcrafted features or graph kernels suffer from high computational complexity or limited expressiveness [6], [7]. Now recent approaches automatically learn graph representations and can be categorized into unsupervised [1], [2] and supervised techniques [3].

Despite their success, supervised methods require labeled data and often involve higher computational costs due to iterative training. While unsupervised methods are often more efficient and label free. However, comparisons between these two remain limited, especially on small benchmark datasets where robustness and efficiency are critical [9], [10]..

In this work, we focus on EMB1 and we compare unsupervised and supervised graph embedding methods on small benchmark datasets from TUDataset. We evaluate Graph2Vec, NetLSD and GIN under a unified experimental franework, focusing not only on classification performance but also on clustering quality, computational cost and stability to structural perturbations.

Our contributions are summarized as follows:

- A comparative study of unsupervised and supervised graph embedding methods on small datasets
- An analysis of performance–efficiency trade offs, and
- An evaluation of embedding robustness under structural perturbations.

II. Related Work

III. Methods

This section describes the graph embedding methods evaluated in this work.

A. Graph2Vec

Starting with Graph2Vec [1], this is an unsupervised graph embedding method that learns a fixed length vector representation in a way analogous to how Doc2Vec learns embeddings for documents. So it's core idea is a graph treated like a document and the rooted subgraphs within it (meaning the local neighborhoods around nodes) are treated like words. The logic is that if two graphs share similar structural patterns their embeddings will be very close in the learned vector space.

The Graph2Vec's algorithm is shown below:

So analytically, Graph2Vec first applies the Weisfeiler-Lehman (WL) relabeling process in order to extract rooted subgraphs around each node up to a specified depth D (Algorithm 2). Then, it creates a vocabulary called *bag of subgraphs* containing all rooted subgraphs. This vocabulary is formed by collecting the multiset of rooted

Algorithm 1 Graph2Vec

Require: Graph set $\mathcal{G}$, WL depth D, dim δ, epochs e, lr α
Ensure: Graph embeddings $\Phi \in \mathbb{R}^{|\mathcal{G}| \times \delta}$

Initialize $\Phi \sim \mathcal{U}(-0.5/\delta, 0.5/\delta)$
for *epoch* $= 1$ to e **do**
  Shuffle $\mathcal{G}$
  for each $G_i \in \mathcal{G}$ **do**
    for $d = 0$ to D **do**
      for each node $n \in G_i$ **do**
        $sg \leftarrow \text{WLSubgraph}(n, G_i, d)$
        Update Φ_i via Skip-Gram(sg, α)
      end for
    end for
  end for
end for
return Φ

Algorithm 2 WLSubgraph

Require: Node n, graph $G = (N, E, \lambda)$, depth d
Ensure: Rooted subgraph label sg

$sg \leftarrow \lambda(n)$
for $i = 1$ to d **do**
  $sg \leftarrow \text{hash}\big(sg \,\|\, \{\lambda(u) : u \in \mathcal{N}(n)\}\big)$
end for
return sg

Algorithm 3 NetLSD (Heat Trace Signature)

Require: Graph $G = (V, E)$, times $T = \{t_k\}_{k=1}^{K}$, Laplacian type $\mathcal{L}$, eigenvalues m
Ensure: Graph signature $s \in \mathbb{R}^K$

Build adjacency A and degree D
$L \leftarrow \begin{cases} I - D^{-1/2} A D^{-1/2}, & \mathcal{L} = \text{norm} \\ D - A, & \mathcal{L} = \text{comb} \end{cases}$
Compute smallest m eigenvalues $\{\lambda_i\}_{i=1}^{m}$ of L
for $k = 1$ to K **do**
  $s_k \leftarrow \sum_{i=1}^{m} e^{-t_k \lambda_i}$
end for
$s \leftarrow \log(s)$ ▷ optional
$s \leftarrow \text{Normalize}(s)$ ▷ optional
return s

Algorithm 4 GIN Forward Pass

Require: Graph $G = (V, E)$, node features $\{h_v^{(0)}\}$, layers L
Ensure: Graph embedding z_G

for $k = 1$ to L **do**
  for all $v \in V$ **do**
    $h_v^{(k)} \leftarrow \text{MLP}^{(k)}\Big((1 + \epsilon^{(k)}) h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)}\Big)$
  end for
end for
$z_G \leftarrow \text{READOUT}(\{h_v^{(L)}\})$
return z_G

subgraphs obtained across WL iterations for all graphs. And finally, it employs a skipgram model to learn graph embeddings by maximizing the likelihood of observing the subgraphs within each graph (Algorithm 1).

B. NetLSD

NetLSD [2] represent a given graph based on the spectrum of the graph Laplacian capturing global stactural information. Its main idea is that every graph has a Laplacian matrix whose eigenvalues encode important structural information. Instead of directly using the raw eigenvalues, it summarizes them using diffusion process, specifically the Heat Kernel.

The NetLSD's algorithm is shown below:

The key advantage of this method lies in its permutation invariance and scalability as we will explore and confirm later.. To improve computational efficiency, it approximates the Laplacian eigenvalue distribution using stochastic trace estimation and so it avoids full eigendecomposition. This hepls NetLSD to maintain stable performace even for large graphs.

C. GIN

The Graph Isomorphism Network (GIN) [3], known as a supervised graph neural network, is designed to maximize discriminative power among GNNs. GIN's node representation are updated by aggregating context from neighbor nodes. At each layer, node features are updated using a sum aggregation function followed by a multilayer perceptron (MLP), allowing the model to distinguish different graph structures effectively as shown in Algorithm 4. Node embeddings are then aggregated using a global pooling operation to produce a graph representation. This representation is then passed to a classifier and trained end-to-end using labeled data as shown in Algorithm 5.

In contrast to unsupervised methods, GIN learns task specific embeddings optimized directly for classification accuracy. While this leads to great performances, it requires labeled datasets (which is difficult to obtain) and needs higher computational cost because it's iterative training.

It's algorithm is shown below:

IV. Datasets

The experiments in this work are conducted on three datasets from TUDataset: MUTAG, ENZYMES, and IMDB-MULTI. These datasets are small but diverse since they cover chemistry, biology and social networks. They are standard benchmarks used in the graph representation learning literature [1]–[3] for evaluating graph embedding and classification methods.

A. MUTAG

MUTAG [4] is a molecular graph dataset consisting of chemical compounds where each graph represents a molecule, nodes correspond to atoms and edges represent chemical bonds between nodes/atoms. The task is to classify molecules into two classes according to their mutagenic effect on a bacterium. The dataset contains a small number of graphs with discrete node labels representing atom types. Graphs in MUTAG are known to be relatively small and sparse, so it is usefull for fast experimentation.

B. ENZYMES

The ENZYMES [5] dataset is derived from protein tertiary structures and contains graphs representing enzymes.

Algorithm 5 Training GIN

Require: Dataset $\mathcal{D}$, epochs E, lr α, batch size B
Ensure: Trained parameters Θ
Initialize Θ
for *epoch* $= 1$ to E **do**
  for mini-batch $\mathcal{B} \subset \mathcal{D}$ **do**
    for all $(G, y) \in \mathcal{B}$ **do**
      $z_G \leftarrow \text{GINForward}(G)$
      $\hat{y} \leftarrow \text{Classifier}(z_G)$
    end for
    $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|} \sum \ell(\hat{y}, y)$
    Update Θ (e.g., Adam)
  end for
end for
return Θ

Nodes correspond to secondary structure elements like amino acids and edges indicate spatial proximity or interactions between residues. Each graph is labeled according to the enzyme's functional class and as a result we have a multi class classification task, specifically 6 enzyme classes.

C. IMDB-MULTI

MDB-MULTI [5] is a social network dataset constructed from movie collaboration data. Each graph represents the ego network of actors appearing in a movie, nodes represent actors and edges show the coappearance relationships in the same movie. Unlike molecular datasets, IMDB-MULTI does not provide node labels or attributes.

V. Experiments and Evaluation

A. Evaluation Protocol for Graph Classification

We evaluate the graph embeddings using the three TUDataset as we mentioned earlier. Methods Graph2Vec and NetLSD are proccesses under the same pipeline for the classification, while supervised GIN follows its own supervised training protocol for the classification task. Performance for this task is reported using Accuracy, F1 score and AUC with extra computational measurments such as training time, generation time and peak tracemalloc mb, rss before mb, rss after mb, meaning memory usage during training. In addition we study the influence of different embedding dimensionality evaluating on the different dim sizes as we mentioned in earlier section (64, 128 and 256). We thought to use smaller dimensions to reduce computational cost and memory usage, but we observed that with higher dims, we get better results as we will see in the results section.

B. Classification Using Unsupervised Embeddings

For each dataset and embedding dim we load each graph's embedding from CSV files. Each CSV contains one row per graph with a label columb and d embedding feature columns. For each configuration, meaning dataset and dimension, we split our embeddings into 80% train and 20% test.

Before training any classifier, embeddings are standardized using sklearn's StandardScaler. This step is critical when training to distance and margin based classifier, like SVM, MLP etc., and ensures a consistent scale across all features. Since Graph2Vec and NetLSD are unsupervised, their embeddings are evaluated by training standard classifiers on top of the learned graph vectors. We choose 5 different classifiers:

- Support Vector Machine (SVM) with RBF kernel and probability calibration enabled. for AUC computation,
- Multi layer Perceptron (MLP), whose hidden layer sizes are adapted based on embedding dimensionality. We chose for larger embeddings to use deeper networks,
- Random Forest Classifier with with the number of trees and maximum depth adapted to dataset size to mitigate overfitting on small datasets,
- Logistic Regression with multinomial option for multiclass datasets, and
- Gradient Boosting with moderate depth

In addition, we combine Random Forest, Gradient Boosting and SVM to observe if we'll get better classification results by ensembling different classifiers. We named this combination as *Voting Classifier*. All results are shown and discussed in section *VI*.

C. Supervised Classification

Unlike Graph2Vec and NetLSD, GIN is trained in a supervised manner. Our implementation uses PyTorch Geometric and follows the architecture and training protocol described in Algorithms 4 and 5. So in contrast with the unsupervised methods, we don't use 5 different classifiers on top of GIN embeddings, because GIN itself is already a classifier. We computed the graph embeddings by training GIN and in parallel we optimize the loss for the classifiication metrics. As will be shown in the experimental results, this supervised training allows GIN to outperform the unsupervised methods on graph classification tasks.

The GIN model consists of stacked GINConv layers, each parameterized by an MLP, followed by batch normalization, ReLU activation, dropout and a residual connection. This design helps the model to learn complex graph structures while avoiding overfitting. To further enhance representational capacity, we employ jumping Knowledge with concatenation allowing the final graph embedding to incorporate information from all intermediate layers.

To improve classification results we add several additional design choices and training options. First, we run our experiments enabling -gfeat option, which concatenates simple graph level statistics (like the number of nodes, edges and graph density) to the learned embedding. Second, we use additive global pooling (–pool add), which is known to provide stable performance across datasets. Thrid, we apply feature normalization (–norm_feats) after constructing node features and we observed that the optimization behavior improved. Finally, we apply label smoothing during training with value

0.05 (–label_smoothing 0.05) in the cross entropy loss to reduce overconfidence and improve generalization on small datasets.

To further study the effect of model capacity and training configuration, we vary multiple GIN hyperparameters during experimentation. Like the unsupervised methods, we evaluate GIN with embedding dimensions of 64, 128 and 256. We, also, trained for 200, 300, 500 and 800 epochs, depending on the dataset, but it is right to mention that this was a little bit overkill and that would be better if we could stick to better difference 50-200 epochs. Having, also, early stopping made the different epochs less critical. We explore, also, two learning rates $1 \cdot 10^{-3}$ and $5 \cdot 10^{-4}$, which are commonly used for training. Dropout is applied with rates of 0.0 and 0.4 to assess the impact of regularization on small datasets. For IMDB-MULTI, which contains larger graphs but no node attributes, we use a batch size of 64 (the same for MUTAG, while ENZYMES's batxhsize is set to 32), while the number of GIN layers is set to 3, reflecting the shallower architecture required for social network graphs. The number of layers for MUTAG is set to 5 and for ENZYMES to 6.

This hyperparameter exploration allows us to analyze how architectural depth, embedding dimensionality and regularization affect both classification accuracy and the quality of the learned graph embeddings. The results of these experiments are presented and discussed in section *VI.*

D. Clustering Evaluation

E. Clustering for Unsupervised Embeddings

F. Clustering for Supervised Embeddings

G. Stability Analysis

Because real world applications involve noisy data, for example edges may be missing and data in general may be corrupted, it is important to evaluate the robustness of our graph embeddings to such perturbations. So, we compute a stability analysis by introducing random permutations to the graph structures and node features.

For each dataset, we generated perturbed versions by applying 2 types of permutations: First, we apply edge pertubation, removing 10% of existing edges and add the same amount of new random edges. And secondly, we shuffle node attributes by randomly swapping features between pairs of nodes within each graph.

After generating these new datasets, we recompute embeddings using Graph2Vec, NetLSD and GIN with the same scripts and configurations as before. This ensures that any observed changes in the embeddings are solely due to the introduced perturbations and so we can fairly and safely make our comparisons between original and perturbed embeddings. To quality how much the embeddings have changed, we use two complementary measures:

1) Cosine Similarity: For each graph, we compute the cosine similarity between its original embedding and the perturbed embedding. When we receive values close to 1, then the embeddings are very similar to the original one.

$$\cos(z_i, z_i') = \frac{z_i^T \cdot z_i'}{||z_i||_2 \cdot ||z_i'||_2}$$

2) Euclidean Distancw (L2 norm):

$$||z_i - z_i'||_2$$

where smaller distance means higher similarity between original and perturbed embeddings.

TABLE I: GIN classification performance on EMB1 datasets (epochs $\in \{200, 300\}$). Best configuration per embedding dimension is reported.

Dataset	Dim	Epochs	Acc	F1	AUC
MUTAG	64	200	0.842	0.825	0.885
	128	200	0.842	0.808	0.885
	256	200	0.895	0.864	0.859
ENZYMES	64	300	0.433	0.428	0.703
	128	300	0.483	0.492	0.744
	256	300	0.617	0.615	0.809
IMDB-MULTI	64	200	0.500	0.496	0.686
	128	200	0.500	0.486	0.678
	256	200	0.507	0.478	0.714

For each dataset, method and config, we report mean, median, and standard deviation of cosine similarity and L2 distance and then we compare its method in order to obseved somehow any difference in the stabilization logic. In general, cosine similarity is usually better for stability. It measures directional consistensy, while L2 mixes direction and scale. Two embeddings can be semantically indentical but far apart in L2.

After computing the above stability analysis, we further evaluate the practical impact of graph perturbations by repeating the classification and clustering experiments on the perturbed datasets with the same scripts and configs as we mentioned in the previous sections. Every result is reported and discussed in section *VI.*

VI. Results

A. Before Permutation

For the graph classification tasks before any pertubation, we present the performance of each embedding method across the three TUDatasets with accuracy, F1-score and AUC metrics for each method in Tables I, II and III. We preferered to show only the best test configuration per dataset x dim results and for the epochs 200 and 300 (for GIN) for the reason we mentioned in earlier section.

For the MUTAG, we can see that GIN excels in classification accuracy, but NetLSD provides superior probabilistic separation as indicated by its higher AUC scores. AUC slightly decreases with increasing dimensions for GIN, showing that higher classification accuray does not always mean that the method has confidence in its predictions. So, our conclusion is that spectral info captured by NetLSD is beneficial for this dataset. For ENZYMES, the gap

TABLE II: Best Graph2Vec classification results per dataset and embedding dimension. For each dataset–dimension pair, the classifier with the highest test accuracy is reported.

Dataset	Dim	Model	Acc	F1	AUC
MUTAG	64	RF	0.711	0.656	0.775
	128	RF	0.763	0.754	0.797
	256	RF	0.737	0.731	0.745
ENZYMES	64	Ens	0.258	0.252	0.571
	128	RF	0.225	0.219	0.600
	256	RF	0.317	0.310	0.591
IMDB-MULTI	64	RF	0.437	0.435	0.601
	128	GB	0.427	0.426	0.583
	256	SVM	0.433	0.426	0.599

TABLE III: Best NetLSD classification results per dataset and embedding dimension. For each dataset–dimension pair, the classifier with the highest test accuracy is reported.

Dataset	Dim	Model	Acc	F1	AUC
MUTAG	64	Ens	0.895	0.892	0.908
	128	Ens	0.868	0.867	0.883
	256	LR	0.868	0.867	0.929
ENZYMES	64	RF	0.300	0.302	0.666
	128	RF	0.308	0.312	0.653
	256	RF	0.325	0.324	0.673
IMDB-MULTI	64	SVM	0.480	0.477	0.625
	128	RF	0.463	0.465	0.633
	256	RF	0.457	0.456	0.648

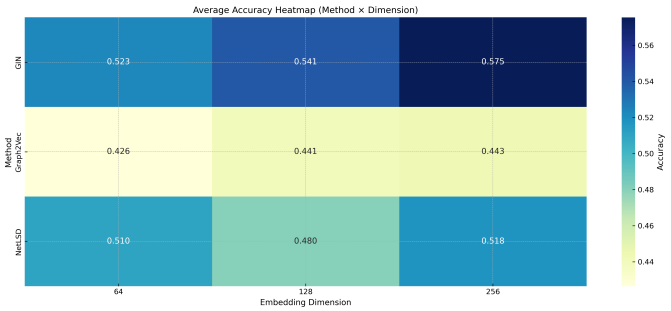


Fig. 1: Average accuracy heatmap across methods and embedding dimensions.

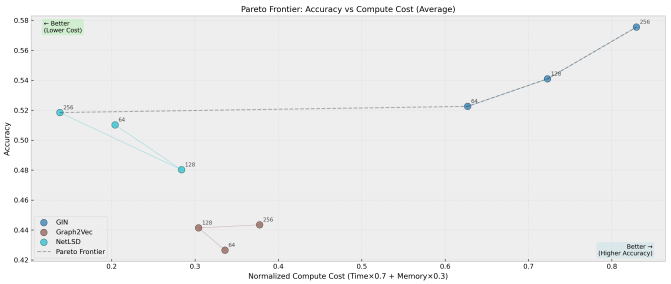


Fig. 2: Pareto frontier illustrating the trade-off between accuracy and normalized compute cost.

between GIN and the other two methods is clearer in all metrics. GIN clearly dominates here and AUC metric confirms that GIN is more confident in its predictions. For IMDB-MULTI, when node features are absent, GIN and NetLSD have similar behavior, with GIN slightly ahead.

So overall, GIN is the most effective method and NetLSD is second best, while Graph2Vec shows the weakest performance across all datasets and metrics. The results confirm that supervised GNNs provide the highest predictive power, while spectral descriptors such as NetLSD remain surprisingly strong for unsupervised graph representation learning.

The experimental results reveal clear trade offs between classification accuracy and computational efficiency across GIN, Graph2Vec and NetLSD, as illustrated in Figures 1-3. All figures are computed with the average results across the three dimensions and three datasets. We chose these 3 images (from the 39 we generated for the classification section) to summarize and make our conclusions.

Figure 1 presents a heatmap of average accuracy across methods and embedding dimensions. We can confirm that GIN consistently outperforms the other two methods across all dimensions.

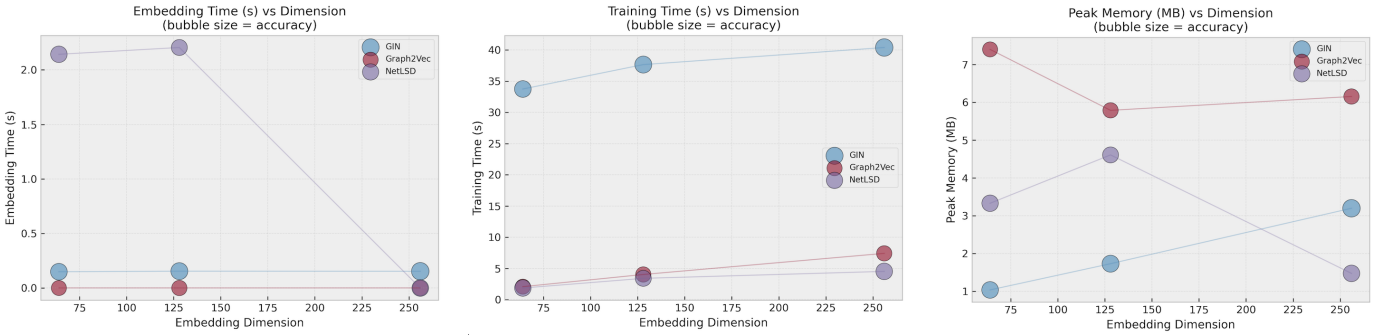
The Pareto frontier in Figure 2 highlights the balance between efficiency and performance. NetLSD lies on the left most side of the optimal region, meaning that it achieves reasonable accuracy with minimal compute cost. GIN dominates with its accuracy, but with a significantlt higher compute cost. Graph2Vec is at the lower part of the graph, showing both lower accuracy from both other methods and higher compute cost, than NetLSD. From the same figure, we can see that GIN benefits monotonically from increased embedding dimensionality, achieving the highest average accuracy at 256 dimensions. In contrast, Graph2Vec and NetLSD reach their peak accuracy at 128 dimensions, beyond which performance declines, indicating diminishing returns with higher dimensions.

Figure 3 breaks down the compute cost into embedding time, training time and peak memory usage. Training time (Figure 3b) is dominated by GIN and increases substantially with embedding dimension. Embedding time (Figure 3a) is highest for NetLSD at lower dimensions due to its spectral computations but decreases at higher dim sizes. Peak memory usage (Figure 3c) is highest for Graph2Vec, while NetLSD remains more memory efficient.

Tables IV, V and VI are a compact version from the original clustering results tables. Best metrics are chosen based on highest ACC and silhouette as a tiebreaker.

Graph2Vec shows the weakest clustering performance. Something we expected based on its classification results and its confidence in seperations (as seen from AUC scores). Wwith the low ACC scores and zero or negative ARI scores, we can say that Graph2Vec has poor aligment with the true class labels.

NetLSD has the strongest clustering performance among the three methods. For MUTAG, it achieves high ACC and solid NMI and ARI scores indicating good alignment with true labels. For ENZYMES and IMDB-



(a) Embedding time (b) Training time (c) Peak memory

Fig. 3: Compute cost components as a function of embedding dimension. Bubble size indicates average accuracy.

TABLE IV: Best Graph2Vec clustering per dataset and dimension (highest ACC; ties by Silhouette).

Data	Dim	Alg	NMI	ARI	ACC	Sil
MUTAG	64	spec	0.048	-0.019	0.532	0.285
	128	km	0.033	-0.019	0.521	0.305
	256	spec	0.037	-0.027	0.511	0.315
ENZ	64	km	0.027	0.006	0.223	0.205
	128	km	0.030	0.010	0.220	0.283
	256	spec	0.021	0.005	0.220	0.210
IMDB	64	km	0.013	0.011	0.402	0.506
	128	km	0.011	0.010	0.399	0.552
	256	km	0.011	0.010	0.400	0.584

TABLE V: Best NetLSD clustering per dataset and dimension (highest ACC).

Data	Dim	Alg	NMI	ARI	ACC	Sil
MUTAG	64	km	0.247	0.302	0.777	0.576
	128	km	0.247	0.302	0.777	0.576
	256	km	0.247	0.302	0.777	0.576
ENZ	64	spec	0.038	0.014	0.250	0.168
	128	spec	0.038	0.014	0.250	0.168
	256	spec	0.038	0.014	0.250	0.168
IMDB	64	km	0.021	0.016	0.410	0.546
	128	km	0.021	0.016	0.410	0.546
	256	km	0.021	0.016	0.410	0.546

TABLE VI: Best GIN clustering per dataset and dimension (highest ACC).

Data	Dim	Alg	NMI	ARI	ACC	Sil
MUTAG	64	km	0.244	0.236	0.745	0.445
	128	km	0.209	0.176	0.713	0.406
	256	km	0.298	0.135	0.691	0.621
ENZ	64	spec	0.082	0.028	0.282	0.029
	128	spec	0.088	0.014	0.288	0.050
	256	spec	0.221	0.128	0.365	0.076
IMDB	64	spec	0.014	0.008	0.378	-0.535
	128	spec	0.011	0.004	0.379	-0.308
	256	spec	0.010	0.006	0.391	-0.245

TABLE VII: GIN embedding stability (best per dataset–dim by highest cosine mean, dropout = 0.0)

Data	D	LR	Cos	σ	L2	σ
MUTAG	64	1e-3	0.918	0.027	256	131
	128	1e-3	0.862	0.032	454	197
	256	1e-3	0.869	0.032	783	381
ENZ	64	5e-4	0.817	0.041	1312	1965
	128	5e-4	0.783	0.040	1100	813
	256	5e-4	0.796	0.038	1804	1743
IMDB	64	5e-4	0.664	0.070	117	160
	128	5e-4	0.671	0.072	138	259
	256	1e-3	0.666	0.066	182	363

MULTI, it consistently outperforms Graph2Vec, though the overall clustering quality is moderate.

GIN creates mixed feelings here. The fact that is a supervised representation makes it not optimal for clustering tasks, meaning for unsupervised tasks. GIN performs better than Graph2Vec, but worse than NetLSD.

B. After Permutation

So after the permutations we did where we explain in detail in the section *V*, we present the stability results for each method in Tables VII, VIII IX and X.

The stability behavior of GIN is strongly influenced by training configuration and dataset characteristics. When we optimize for max cosine (closer to value 1), we take models with dropout 0.0 and we have high cosine similarity values (0.66-0.91 range) with gradually decreasing cosine similarity as dimensionality increases. However, the corresponding L2 distances are relatively large (ranging from 100 to 1800), meaning significant magnitude changes in embeddings despite relatively preserved directions.

When we optimize for min L2 (closer to value 0), we take models with dropout 0.4 and we have lower cosine similarity values (0.12-0.77 range), indicating more directional changes. However, the L2 distances are much smaller (ranging from 66 to 483), showing reduced magnitude changes. This inverse relationship between cosine similarity and L2 distance across configurations highlights the trade off between preserving direction versus magnitude in GIN embeddings under perturbations.

Graph2Vec shows strong stability across all datasets and dimensions. Cosine similarity has closer to 1 values and l2 distances are very small. This indicates that Graph2Vec embeddings are robust to structural and feature pertur-

TABLE VIII: GIN embedding stability (best per dataset–dim by lowest L2 mean, dropout = 0.4).

Data	D	LR	L2	σ	Cos	σ
	64	5e-4	168	43	0.775	0.016
MUTAG	128	1e-3	263	77	0.781	0.030
	256	5e-4	455	167	0.752	0.028
	64	5e-4	402	457	0.563	0.056
ENZ	128	1e-3	483	1835	0.297	0.085
	256	1e-3	460	1579	0.126	0.080
	64	1e-3	69	118	0.663	0.066
IMDB	128	1e-3	105	182	0.625	0.068
	256	1e-3	66	132	0.126	0.072

TABLE IX: Graph2Vec embedding stability under perturbations.

Data	D	Cos	σ	L2	σ
	64	0.899	0.018	0.133	0.052
MUTAG	128	0.910	0.023	0.095	0.039
	256	0.899	0.027	0.068	0.027
	64	0.825	0.029	0.603	0.138
ENZ	128	0.863	0.022	0.545	0.124
	256	0.896	0.020	0.487	0.117
	64	0.719	0.062	0.672	0.325
IMDB	128	0.696	0.046	0.680	0.317
	256	0.661	0.025	0.691	0.323

bations, maintaining both direction and magnitude.

Finally, NetLSD shows mixed stability results. We observe that with dim sizes 256, we have high cosine similarity values and in general small L2 distances (if we compare with GIN). However, at lower dimensions (64 and 128), cosine similarity drops significantly (except for ENZYMES), even becoming negative for MUTAG. L2 distances increase with dimensionality but remain moderate relative to GIN.

So, across the three methods, Graph2Vec is the king of stability, NetLSD achieves very strong stability only at higher dim sizes and GIN shows the most sensitivity to perturbations.

Executing again the classification task after perturbing the graphs ....

TABLE X: NetLSD embedding stability under perturbations.

Data	D	Cos	σ	L2	σ
	64	-0.818	0.366	52.3	30.2
MUTAG	128	-0.818	0.366	52.3	30.2
	256	0.875	0.029	216	58.7
	64	0.942	0.169	26.8	30.6
ENZ	128	0.942	0.169	26.8	30.6
	256	0.964	0.013	104	66.4
	64	0.658	0.566	58.8	61.6
IMDB	128	0.658	0.566	58.8	61.6
	256	0.962	0.027	127	83.9

References

[1] A. Narayanan, M. Chandramohan, L. Chen, Y. Liu, and S. Saminathan, "graph2vec: Learning Distributed Representations of Graphs," *arXiv preprint arXiv:1707.05005*, 2017.

[2] A. Tsitsulin, D. Mottin, P. Karras, A. Bronstein, and E. Müller, "NetLSD: Hearing the Shape of a Graph," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '18)*, 2018.

[3] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful are Graph Neural Networks?" in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[4] A. K. Debnath, R. L. López de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch, "Structure–activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. Correlation with molecular orbital energies and hydrophobicity," *Journal of Medicinal Chemistry*, vol. 34, no. 2, pp. 786–797, 1991, doi: 10.1021/jm00106a046.

[5] R. A. Rossi and N. K. Ahmed, "The Network Data Repository with Interactive Graph Analytics and Visualization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.

[6] N. Shervashidze, P. Schweitzer, E. J. van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler–Lehman Graph Kernels," *Journal of Machine Learning Research*, vol. 12, pp. 2539–2561, 2011.

[7] S. V. N. Vishwanathan, N. N. Schraudolph, R. Kondor, and K. M. Borgwardt, "Graph Kernels," *Journal of Machine Learning Research*, vol. 11, pp. 1201–1242, 2010.

[8] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The Graph Neural Network Model," *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.

[9] F. Errica, M. Podda, D. Bacciu, and A. Micheli, "A Fair Comparison of Graph Neural Networks for Graph Classification," in *Proceedings of the ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[10] V. P. Dwivedi, C. K. Joshi, T. Laurent, Y. Bengio, and X. Bresson, "Benchmarking Graph Neural Networks," *arXiv preprint arXiv:2003.00982*, 2020.

[11] B. Rozemberczki, O. Kiss, and R. Sarkar, "Karate Club: An API Oriented Open-Source Python Framework for Unsupervised Learning on Graphs," in *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*, 2020.

[12] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in *Proceedings of the ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.